Statistical Machine Learning Final Project: Predictions of Flight Delays at the Pittsburgh Airport

Ellie Huang (elliehua) Nikita Jayaprakash (njayapra)

Executive summary

1 Introduction

The goal of this analysis is to predict whether or not a departing flight from Pittsburgh International Airport (airport code PIT) will be delayed based on only information available before the departure of the flight. The dataset is flight information from 2023 - 2025 extracted from the Airline On-Time Performance Data from the Bureau of Transportation Statistics of the U.S. Department of Transportation. In addition to metrics about flight delays, it also reflects commercial flight activity to and from PIT. Feature engineering was used to develop additional metrics based on initial exploratory data analysis. This analysis utilizes various machine learning methods in order to attempt this binary classification on a subset of the 2025 flights, including logistic regression (with and without LASSO regularization), XGBoost, random forest (with and without rebalancing), and an ensemble method that took an average of the predictions of these metrics (with and without weighting different methods). The metrics of model performance are training set misclassification rate, testing set misclassification rate, and area-under-the-curve (AUC). Based on the findings, ... [**insert conclusions here**].

2 Exploration

2.1 Exploratory Data Analysis

Due to the data imbalance in Table 1, where there are more on-time flights than delayed, this analysis will need to adjust the threshold that a successful predictive models performs at.

From Figure 1, delay rates vary meaningfully across carriers, so carrier should be a useful predictor. At the same time, some carriers have much lower flight counts at PIT than others, so raw group means will be noisy. That motivates using smoothed historical delay rates later in feature engineering instead of unsmoothed averages.

Table 1: The departure delay distribution is very imbalanced with some missing values, which may indicate cancelled flights.

	Flight Outcome	Count
	On Time	59534
	Delayed	10972
	NA	950
Total	—	71456

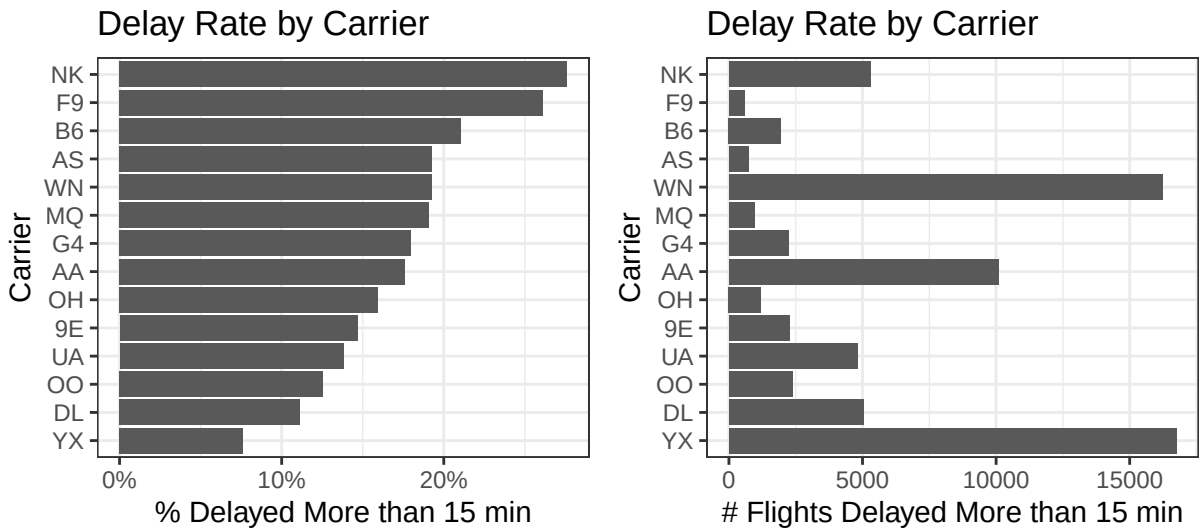

Figure 1: Delay rates and delay counts vary across different carriers, but the carriers with the highest delay rates don't necessarily have the highest delay counts.

Figure 2 shows that early flights start the day with fewer disruptions from previous legs, while later flights are more exposed to effects from earlier problems. This makes scheduled departure hour one of the strongest single predictors in the dataset.

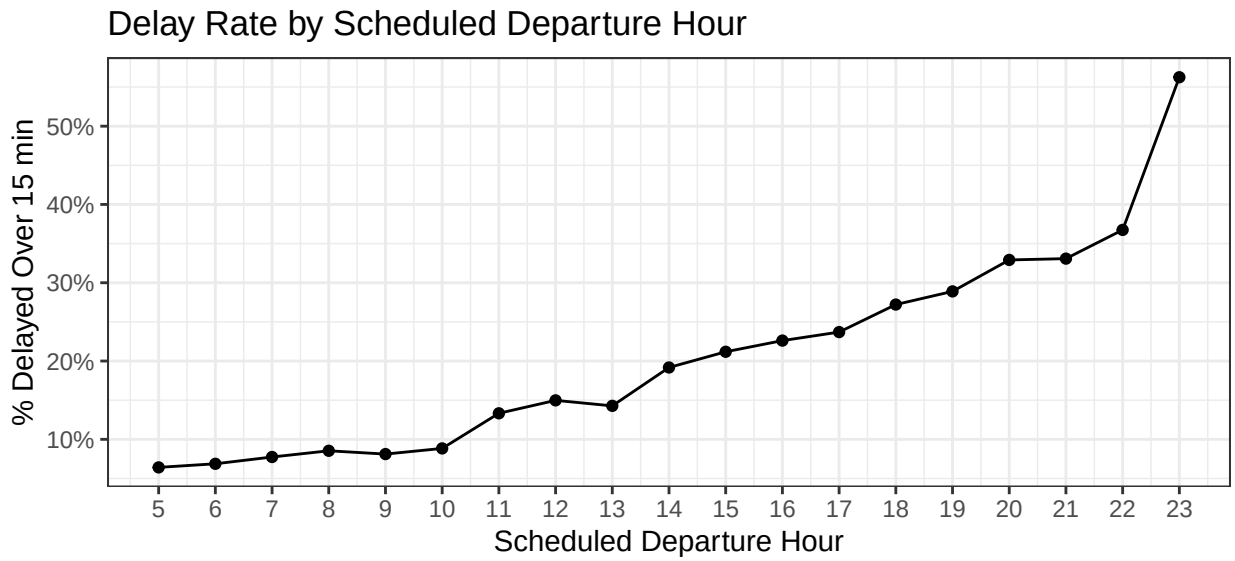

Figure 2: Delay rates rise steadily over the course of the day and are highest in the evening

Figure 3 show s that elays are worse in the summer months and in December, which is consistent with heavier travel volume and more weather-related disruption. Since these seasonal peaks are not spread evenly across the month, this supports adding a simple calendar feature rather than relying only on month.

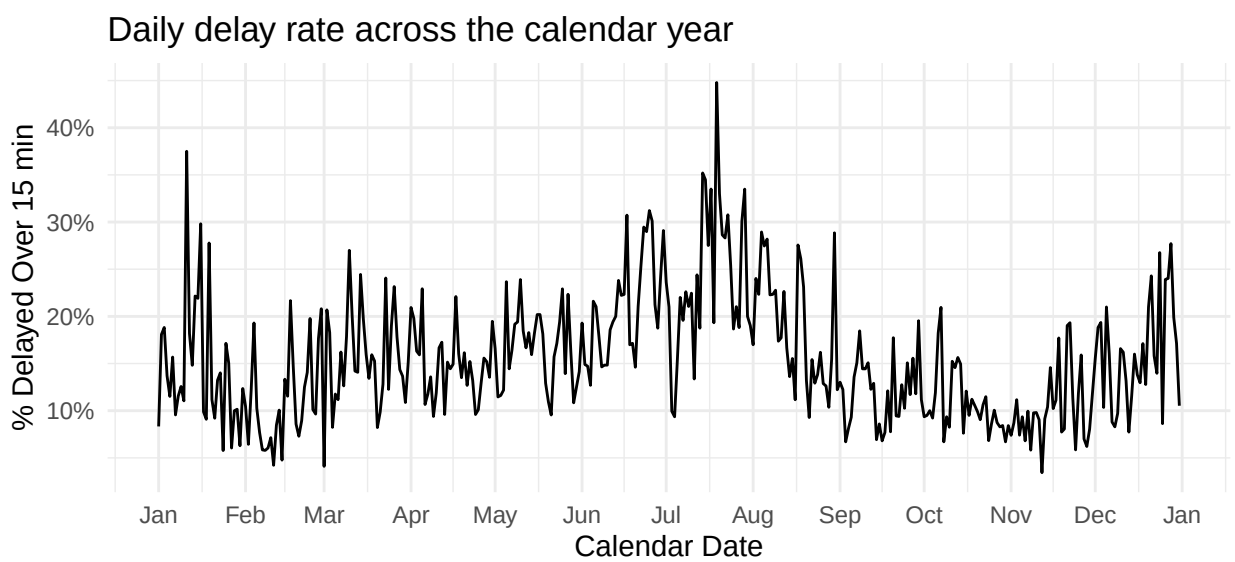

Figure 3: There are seasonal peaks, but the delays are not spread evenly across the month.

Further investigating specific calendar days with high delay rates, Table 2 suggests that there are broader operating shocks affecting many flights at once. That motivates adding dynamic same-day features based on earlier visible flights, along with a historical calendar-day disruption feature learned from the training data.

Figure 4 shows that delay rates differ across destinations, even among the busiest PIT routes, which suggests that route context adds information beyond carrier and time of day alone. Longer routes also show higher delay rates on average, so distance should remain in the model and may be easier to capture through both a raw continuous version and a bucketed factor version.

Table 2: A small number of days have unusually high delay rates and delay counts.

Flight Date	Delay Rate	Count
January 11	0.3750000	176
August 04	0.2894737	190
August 06	0.2819149	188
August 05	0.2746114	193
March 10	0.2700000	200
July 12	0.2440191	209
August 02	0.2401961	204
May 10	0.2390244	205
May 05	0.2367150	207
July 01	0.2361809	199

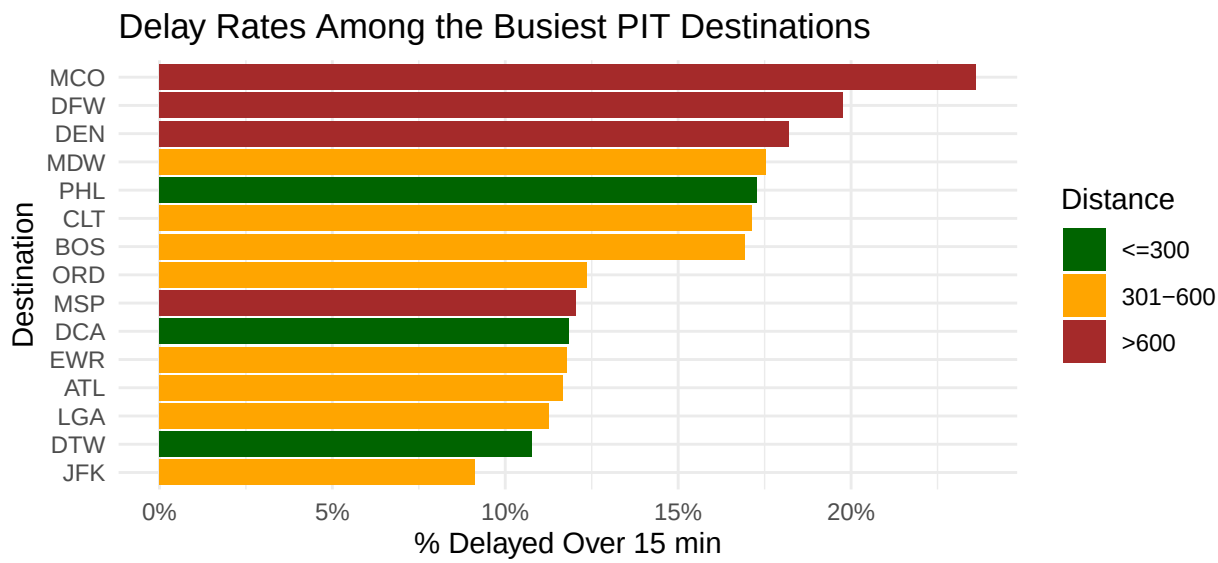

Figure 4: Longer routes and the specific destinations tend to have higher delay rates.

2.2 Dimension Reduction

This analysis applied principal component analysis (PCA) on a subset of various that were considered important to predicting delay through the previous EDA. These variables were month, day of the month, day of the week, the carrier, the distance and the destination. As shown in Figure 5, while this does create clusters, these clusters do not seem useful for differentiating delayed and on-time flights, so the principal components will not be used for feature engineering. Feature engineering will be developed based on the intuition built from the exploratory data analysis.

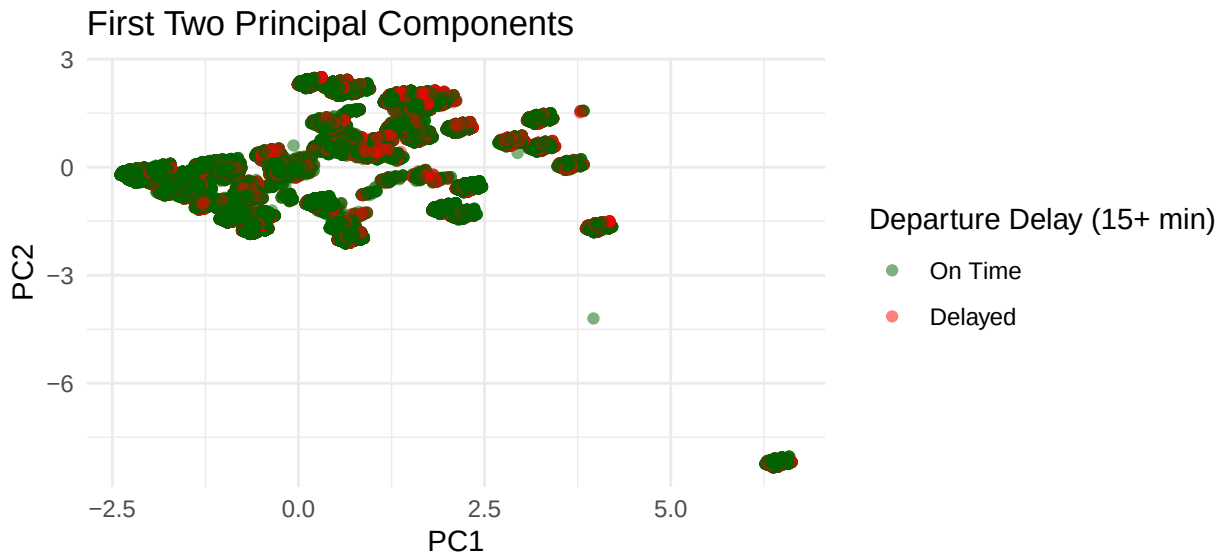

Figure 5: While PCA does create distinct clusters, they do not seem to differentiate delayed status.

3 Supervised Analysis

3.1 Feature Engineering

The EDA suggests that delay risk depends on a mix of stable historical tendencies and day-specific operating conditions. Based on those patterns, we engineer features that capture time-of-day effects, carrier and route history, route distance, seasonality, and same-day disruption.

As shown by Table 3,. . .

Table 3: The following features were engineered based on the EDA results and uniformly plugged into all machine learning methods

Feature	Definition
`sched_dep_hour`, `sched_arr_hour`	
`is_late_arrival`	
`crs_elapsed_time`	
`carrier_size`	
`dest_region`	
`is_hub_dest`	
`distance`, `distance_bucket`	
`month`, `day_of_week`	
`peak_season`	
`is_morning`, `is_evening`	
`carrier_delay_rate_smooth`	
`dest_delay_rate_smooth`	
`route_delay_rate_smooth`	
`prior_same_day_delay_rate`	
`before_today`	
`visible_delay_rate`	
`visible_count`	
`visible_max_delay`	

Feature	Definition
visible_n_delayed	
visible_recent_rate	
visible_severe_delay	
visible_carrier_delay_rate	
visible_carrier_n	
flights_same_hour	
high_disruption_calendar_day	

3.2 Machine Learning Techniques

4 Analysis of Results

4.1 What kinds of flights do you do well or poorly on?

4.2 Predicting Minutes Delayed

4.3 Trade-Off of a Fixed Cost

Appendix

```r
###########################
# STYLE EDITS: IGNORE THIS
###########################

# normally you'll want to include this with the libraries at the beginning of your document
knitr::opts_chunk$set(message = FALSE) # include this if you don't want markdown to knit messages
knitr::opts_chunk$set(warning = FALSE) # include this if you don't want markdown to knit warnings
knitr::opts_chunk$set(echo = FALSE) # set echo = FALSE to hide code from output

library(tidyverse)
library(adabag)
library(caret)
library(gt)
library(e1071)
library(tidymodels)
library(here)
library(janitor)
library(lubridate)
library(pROC)
library(randomForest)
library(skimr)
library(scales)
library(forcats)
library(ranger)
library(patchwork)
library(glmnet)
library(xgboost)
# Load data
flights2023 <- read_csv(here("data", "flights2023.csv")) |>
 clean_names()

flights2024 <- read_csv(here("data", "flights2024.csv")) |>
 clean_names()

test <- read_csv(here("data", "flights2025.csv")) |>
 clean_names()

test_visible <- read_csv(here("data", "flights2025_visible.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

test_guess <- read_csv(here("data", "flights2025_guess.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

# Build training set
train <- bind_rows(
 flights2023 |> mutate(data_year = 2023),
 flights2024 |> mutate(data_year = 2024)
)
# Apply the same cleaning steps to each data frame so train and test sets are consistent
clean_flights <- function(df) {
```

```r
 df |>
 clean_names() |>
 mutate(
 fl_date = as.Date(fl_date),
 dep_del15 = as.numeric(dep_del15),
 op_unique_carrier = as.factor(op_unique_carrier),
 origin = as.factor(origin),
 dest = as.factor(dest),
 sched_dep_hour = floor(crs_dep_time / 100),
 sched_arr_hour = floor(crs_arr_time / 100),
 month = as.factor(month),
 day_of_week = as.factor(day_of_week)
 )
}

train <- clean_flights(train)
test <- clean_flights(test)
test_visible <- clean_flights(test_visible)
test_guess <- clean_flights(test_guess)
# Restrict to departures from PIT
train <- train |>
 filter(origin == "PIT")

test_visible <- test_visible |>
 filter(origin == "PIT")

test_guess <- test_guess |>
 filter(origin == "PIT")
# Inbound flight features
# For each PIT departure, we try to match the same aircraft (tail + date) arriving at PIT
# earlier that day. Match rates: ~55% for training, ~15% for guess flights.
# For guess, only arrivals in flights2025_visible.csv have known arr_delay -
# guess-period arrivals are blanked out. The 85% with no match are overnighted
# or first-flight aircraft, which is itself informative (lower delay risk).
arrivals_train <- bind_rows(
  clean_flights(read_csv(here("data", "flights2023.csv"), show_col_types = FALSE)) |>
    filter(dest == "PIT") |> mutate(data_year = 2023),
  clean_flights(read_csv(here("data", "flights2024.csv"), show_col_types = FALSE)) |>
    filter(dest == "PIT") |> mutate(data_year = 2024)
)

# Only visible arrivals have non-NA arr_delay bc guess-period arrivals are blanked out
arrivals_test_visible <- clean_flights(
  read_csv(here("data", "flights2025_visible.csv"), show_col_types = FALSE)
) |> filter(dest == "PIT") |> mutate(data_year = 2025)

# For each (date, tail_num), keep the latest arriving flight as the inbound leg
make_inbound_lookup <- function(arrivals_df) {
  arrivals_df |>
    filter(!is.na(tail_num), tail_num != "", !is.na(arr_delay)) |>
    group_by(fl_date, tail_num) |>
    slice_max(crs_arr_time, n = 1, with_ties = FALSE) |>
    ungroup() |>
```

```r
    select(fl_date, tail_num,
           inbound_arr_delay = arr_delay,  # minutes late on arrival
           inbound_arr_del15 = arr_del15,  # binary: arrived >= 15 min late
           inbound_crs_arr_time = crs_arr_time)  # scheduled arrival time
}

inbound_train <- make_inbound_lookup(arrivals_train)
inbound_visible <- make_inbound_lookup(arrivals_test_visible)
inbound_guess <- inbound_visible  # guess set uses visible-only lookup

join_inbound <- function(dep_df, inbound_df) {
  dep_df |>
    left_join(inbound_df, by = c("fl_date", "tail_num")) |>
    mutate(
      # min between scheduled inbound arrival and scheduled departure
      turnaround_mins = sched_dep_hour * 60 + (crs_dep_time %% 100) -
                          (inbound_crs_arr_time %/% 100 * 60 + inbound_crs_arr_time %% 100),
      # indicator (< 45 min scheduled buffer is risky)
      tight_turnaround = if_else(!is.na(turnaround_mins) & turnaround_mins < 45, 1L, 0L),
      # indicator (whether we found a matching inbound leg at all)
      # has_inbound = 0 could mean overnighted OR inbound after cutoff in guess set
      # so we keep it as context but don't include it in model_vars
      has_inbound = if_else(!is.na(inbound_arr_delay), 1L, 0L),
      # fill unmatched rows with 0 (no evidence of inbound delay)
      inbound_arr_delay = coalesce(as.numeric(inbound_arr_delay), 0),
      inbound_arr_del15 = coalesce(as.numeric(inbound_arr_del15), 0),
      turnaround_mins = coalesce(turnaround_mins, 0)
    )
}

train <- join_inbound(train, inbound_train)
test_visible <- join_inbound(test_visible, inbound_visible)
test_guess <- join_inbound(test_guess, inbound_guess)
train |>
  mutate(`Flight Outcome` = factor(dep_del15,
                        labels = c("On Time", "Delayed"))) |>
  group_by(`Flight Outcome`) |>
  summarize(Count = n()) |>
  gt() |>
    grand_summary_rows(
    columns = Count,
    fns = list(
      Total = ~sum(.)
    )
  ) |>
  tab_options(
  table.width = pct(60)
) |>
  tab_caption("The departure delay distribution is very imbalanced with some missing values, which may i
carrier_delay_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(op_unique_carrier) |>
 summarise(
```

```r
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(desc(delay_rate))


carrier_perc <- carrier_delay_tbl |>
 ggplot(aes(x = reorder(op_unique_carrier, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay Rate by Carrier",
 x = "Carrier",
 y = "% Delayed More than 15 min"
 )  +
 theme_bw()

carrier_count <- carrier_delay_tbl |>
 ggplot(aes(x = reorder(op_unique_carrier, delay_rate), y = n)) +
 geom_col() +
 coord_flip() +
 labs(
 title = "Delay Rate by Carrier",
 x = "Carrier",
 y = "# Flights Delayed More than 15 min"
 )  +
 theme_bw()

carrier_perc + carrier_count
hour_delay_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 )

hour_delay_tbl |>
 ggplot(aes(x = sched_dep_hour, y = delay_rate)) +
 geom_line() +
 geom_point() +
 scale_x_continuous(breaks = 0:23) +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay Rate by Scheduled Departure Hour",
 x = "Scheduled Departure Hour",
 y = "% Delayed Over 15 min"
 ) +
 theme_bw()
# Delay rate by calendar date, aligned to a common year for display
```

```r
train |>
 filter(!is.na(dep_del15)) |>
 mutate(plot_date = make_date(year = 2010, month = as.integer(as.character(month)), day = day_of_month))
 group_by(plot_date) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = plot_date, y = delay_rate)) +
 geom_line() +
 scale_x_date(date_labels = "%b", date_breaks = "1 month") +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Daily delay rate across the calendar year",
 x = "Calendar Date",
 y = "% Delayed Over 15 min"
 ) +
 theme_minimal()
bad_days_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(fl_date) |>
 summarise(
 `Delay Rate` = mean(dep_del15),
 Count = n(),
 .groups = "drop"
 ) |>
 filter(Count >= 10) |>
 arrange(desc(`Delay Rate`)) |>
  mutate(fl_date = as.character(fl_date))
bad_days_tbl |>
  mutate(`Flight Date` = format(as.Date(fl_date, format = "%m-%d-%y"), "%B %d")
) |>
  head(10) |>
  select(`Flight Date`, `Delay Rate`, Count) |>
  gt() |>
  tab_options(
  table.width = pct(60)
) |>
  tab_caption("A small number of days have unusually high delay rates and delay counts.")
top_dests <- train |>
 count(dest, sort = TRUE) |>
 slice_head(n = 15) |>
 pull(dest)

train <- train |>
 mutate(
  `Distance` = case_when(
  distance > 600 ~ ">600",
  distance > 300 ~ "301-600",
  TRUE ~ "<=300"
 ),
 `Distance` = factor(
 `Distance`,
```

```r
 levels = c("<=300", "301-600", ">600")
 )
 )

train |>
 filter(!is.na(dep_del15), dest %in% top_dests) |>
 group_by(dest, `Distance`) |>
 summarise(
   delay_rate = mean(dep_del15),
   n = n(),
   .groups = "drop"
 ) |>
  arrange(delay_rate) |>
  ggplot(aes(x = reorder(dest, delay_rate), y = delay_rate,
             fill = `Distance`)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = percent_format()) +
  labs(
    title = "Delay Rates Among the Busiest PIT Destinations",
    x = "Destination",
    y = "% Delayed Over 15 min"
    ) +
  theme_minimal() +
  scale_fill_manual(values = c("darkgreen", "orange", "brown"))
train_pca <- train |>
  select(dep_del15, month, day_of_month, day_of_week,
         op_unique_carrier, distance, dest) |>
  drop_na()
train_pca_matrix <- model.matrix(dep_del15 ~ .,
                                 data = train_pca)[, -1]
train_pca_matrix <- train_pca_matrix[, (apply(train_pca_matrix, 2, var) != 0)]
pca_result <- prcomp(train_pca_matrix, center = TRUE, scale. = TRUE)
# Convert PCA scores to a data frame
pca_df <- as.data.frame(pca_result$x)

# Add the outcome variable for coloring
pca_df$dep_del15 <- factor(train_pca$dep_del15,
                           labels = c("On Time", "Delayed"))

# Plot PC1 vs PC2
ggplot(pca_df, aes(x = PC1, y = PC2, color = dep_del15)) +
  geom_point(alpha = 0.5) +
  labs(
    title = "First Two Principal Components",
    x = "PC1",
    y = "PC2",
    color = "Departure Delay (15+ min)"
  ) +
  theme_minimal() +
  scale_color_manual(values = c("darkgreen", "red"))
add_basic_features <- function(df) {
 df |>
```

```r
 mutate(
 peak_season = if_else(as.integer(as.character(month)) %in% c(6, 7, 8, 12), 1, 0),
 is_evening = if_else(sched_dep_hour >= 18, 1, 0),
 is_morning = if_else(sched_dep_hour <= 8, 1, 0),
 distance_bucket = case_when(
 distance > 600 ~ ">600",
 distance > 300 ~ "301-600",
 TRUE ~ "<=300"
 ),
 carrier_hour = paste(op_unique_carrier, sched_dep_hour, sep = "_"),
 # scheduled arrival hour at destination - late-night arrivals face thinner operations
 sched_arr_hour = floor(crs_arr_time / 100),
 is_late_arrival = if_else(sched_arr_hour >= 21 | sched_arr_hour <= 1, 1L, 0L),
 # scheduled flight duration - longer flights have more exposure to en-route disruption
 crs_elapsed_time = as.numeric(crs_elapsed_time)
 )
}

train <- add_basic_features(train)
test_visible <- add_basic_features(test_visible)
test_guess <- add_basic_features(test_guess)
global_delay_rate <- mean(train$dep_del15, na.rm = TRUE)

make_smoothed_rates <- function(df, group_var, target = "dep_del15", m = 100) {
 group_sym <- rlang::sym(group_var)
 target_sym <- rlang::sym(target)

 df |>
 filter(!is.na(!!target_sym)) |>
 group_by(!!group_sym) |>
 summarise(
 n = n(),
 mean_target = mean(!!target_sym),
 .groups = "drop"
 ) |>
 mutate(
 smoothed_rate = (n * mean_target + m * global_delay_rate) / (n + m)
 ) |>
 select(!!group_sym, smoothed_rate, n)
}
# Route = carrier + dest combination
train <- train |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))
test_visible <- test_visible |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))
test_guess <- test_guess |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))

# Carrier and dest rates computed on full train - stable enough that leakage is minimal
carrier_rates <- make_smoothed_rates(train, "op_unique_carrier", m = 200) |>
 rename(carrier_delay_rate_smooth = smoothed_rate, carrier_n = n)

dest_rates <- make_smoothed_rates(train, "dest", m = 200) |>
 rename(dest_delay_rate_smooth = smoothed_rate, dest_n = n)
# OOF (out-of-fold) smoothed rates for important features
# Each row's rate is computed from the other 4 folds only, preventing label leakage
```

```r
set.seed(100)
n_folds <- 5
fold_id <- sample(rep(1:n_folds, length.out = nrow(train)))

make_oof_rates <- function(df, group_var, target = "dep_del15", m = 100) {
  group_sym <- rlang::sym(group_var)
  target_sym <- rlang::sym(target)
  out_col <- paste0(group_var, "_oof_rate")
  global <- mean(df[[target]], na.rm = TRUE)

  result <- numeric(nrow(df))
  for (k in 1:n_folds) {
    in_fold <- which(fold_id == k)
    out_fold <- which(fold_id != k)
    lookup <- df[out_fold, ] |>
      filter(!is.na(!!target_sym)) |>
      group_by(!!group_sym) |>
      summarise(n = n(), mean_t = mean(!!target_sym), .groups = "drop") |>
      mutate(rate = (n * mean_t + m * global) / (n + m))
    merged <- df[in_fold, ] |>
      select(!!group_sym) |>
      left_join(lookup |> select(!!group_sym, rate), by = group_var)
    result[in_fold] <- coalesce(merged$rate, global)
  }
  result
}

# Use stronger smoothing (bigger m) for route and flight level features bc those groups
# are more granular and have fewer observations per group (noisier raw delay rates)
train$carrier_hour_delay_rate_smooth <- make_oof_rates(train, "carrier_hour", m = 100)
train$route_delay_rate_smooth <- make_oof_rates(train, "route", m = 150)

# Week-of-year rate - replaces raw calendar-day rate (~2 obs/day with 2 years of data)
# Week of year gives ~14 observations per cell, same seasonal signal but more stable
train <- train |> mutate(week_of_year = as.character(week(fl_date)))
test_visible <- test_visible |> mutate(week_of_year = as.character(week(fl_date)))
test_guess <- test_guess |> mutate(week_of_year = as.character(week(fl_date)))
train$week_delay_rate_smooth <- make_oof_rates(train, "week_of_year", m = 50)
train$avg_delay_rate_for_calendar_day <- train$week_delay_rate_smooth  # keep for computing high_disrup

# For test sets, use the full-train lookup
carrier_hour_rates <- make_smoothed_rates(train, "carrier_hour", m = 100) |>
  rename(carrier_hour_delay_rate_smooth = smoothed_rate, carrier_hour_n = n)

route_rates <- make_smoothed_rates(train, "route", m = 150) |>
  rename(route_delay_rate_smooth = smoothed_rate, route_n = n)

week_rates <- make_smoothed_rates(train, "week_of_year", m = 50) |>
  rename(week_delay_rate_smooth = smoothed_rate, week_n = n)

bad_day_lookup <- train |>
  filter(!is.na(dep_del15)) |>
  group_by(week_of_year) |>
```

```r
  summarise(avg_delay_rate_for_calendar_day = mean(dep_del15), .groups = "drop")
# Number of flights scheduled out of PIT in the same departure hour
hourly_congestion <- train |>
  group_by(fl_date, sched_dep_hour) |>
  summarise(flights_same_hour = n(), .groups = "drop")

train <- train |>
  left_join(hourly_congestion, by = c("fl_date", "sched_dep_hour"))

# For test sets, use average congestion by hour across training days
# (bc we don't know actual 2025 schedules ahead of time for the guess flights)
avg_hourly_congestion <- train |>
  group_by(sched_dep_hour) |>
  summarise(avg_flights_same_hour = mean(flights_same_hour, na.rm = TRUE), .groups = "drop")

test_visible <- test_visible |>
  left_join(
    test_visible |>
      group_by(fl_date, sched_dep_hour) |>
      summarise(flights_same_hour = n(), .groups = "drop"),
    by = c("fl_date", "sched_dep_hour")
  )

test_guess <- test_guess |>
  left_join(avg_hourly_congestion, by = "sched_dep_hour") |>
  mutate(flights_same_hour = avg_flights_same_hour)
# For train OOF columns already attached above so only need to join carrier/dest rates
# For test sets join all lookup tables from full training data
apply_rate_features <- function(df, is_train = FALSE) {
  df <- df |>
    left_join(carrier_rates, by = "op_unique_carrier") |>
    left_join(dest_rates, by = "dest") |>
    mutate(
      carrier_delay_rate_smooth = coalesce(carrier_delay_rate_smooth, global_delay_rate),
      dest_delay_rate_smooth = coalesce(dest_delay_rate_smooth, global_delay_rate)
    )
  if (!is_train) {
    df <- df |>
      left_join(carrier_hour_rates, by = "carrier_hour") |>
      left_join(route_rates, by = "route") |>
      left_join(week_rates, by = "week_of_year") |>
      left_join(bad_day_lookup, by = "week_of_year") |>
      mutate(
        carrier_hour_delay_rate_smooth = coalesce(carrier_hour_delay_rate_smooth, global_delay_rate),
        route_delay_rate_smooth = coalesce(route_delay_rate_smooth, global_delay_rate),
        week_delay_rate_smooth = coalesce(week_delay_rate_smooth, global_delay_rate),
        avg_delay_rate_for_calendar_day = coalesce(avg_delay_rate_for_calendar_day, global_delay_rate)
      )
  }
  df
}

train_fe <- apply_rate_features(train, is_train = TRUE)
```

```r
test_visible_fe <- apply_rate_features(test_visible, is_train = FALSE)
test_guess_fe <- apply_rate_features(test_guess, is_train = FALSE)
add_running_day_features <- function(df) {
 df |>
 arrange(fl_date, crs_dep_time) |>
 group_by(fl_date) |>
 mutate(
 before_today = row_number() - 1,
 delays_before_today = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
 prior_same_day_delay_rate = if_else(
 before_today > 0,
 delays_before_today / before_today,
 0
 )
 ) |>
 ungroup()
}

train_fe <- add_running_day_features(train_fe)
test_visible_fe <- add_running_day_features(test_visible_fe)
test_guess_fe <- add_running_day_features(test_guess_fe)
# Rich same-day context from visible flights
# The project allows using all visible-window flights to predict guess flights.
# We build summaries from both departures and arrivals in the visible window.

# Overall departure health for the day
vis_dep_summary <- test_visible_fe |>
  filter(!is.na(dep_del15)) |>
  arrange(fl_date, crs_dep_time) |>
  group_by(fl_date) |>
  summarise(
    visible_count = n(),
    visible_delay_rate = mean(dep_del15),
    visible_max_delay = max(dep_delay, na.rm = TRUE),  # flags big disruption events
    visible_n_delayed = sum(dep_del15),
    visible_recent_rate = mean(tail(dep_del15, 5)),  # last 5 flights, more current signal
    .groups = "drop"
  ) |>
  mutate(
    # a single delay > 60 min often signals a ground stop or weather event
    visible_severe_delay = if_else(visible_max_delay > 60, 1L, 0L),
    visible_max_delay = if_else(is.infinite(visible_max_delay), 0, visible_max_delay)
  )

# Per-carrier delay rate from visible flights that day
# e.g. if United has been running 40 min late all morning, that predicts the 7pm United flight
vis_carrier_summary <- test_visible_fe |>
  filter(!is.na(dep_del15)) |>
  group_by(fl_date, op_unique_carrier) |>
  summarise(
    visible_carrier_delay_rate = mean(dep_del15),
    visible_carrier_n = n(),
    .groups = "drop"
```

```r
  )

# Arrival health at PIT from visible window
# planes arriving late become planes departing late
arrivals_vis_summary <- arrivals_test_visible |>
  filter(!is.na(arr_delay)) |>
  group_by(fl_date) |>
  summarise(
    vis_arr_delay_rate = mean(arr_del15, na.rm = TRUE),  # fraction of arrivals that were late
    vis_arr_mean_delay = mean(arr_delay, na.rm = TRUE),  # mean arrival delay in minutes
    .groups = "drop"
  )

# Join all summaries onto guess set
test_guess_fe <- test_guess_fe |>
  left_join(vis_dep_summary, by = "fl_date") |>
  left_join(vis_carrier_summary, by = c("fl_date", "op_unique_carrier")) |>
  left_join(arrivals_vis_summary, by = "fl_date") |>
  mutate(
    visible_count = coalesce(visible_count, 0L),
    visible_delay_rate = coalesce(visible_delay_rate, global_delay_rate),
    visible_max_delay = coalesce(visible_max_delay, 0),
    visible_n_delayed = coalesce(visible_n_delayed, 0L),
    visible_recent_rate = coalesce(visible_recent_rate, global_delay_rate),
    visible_severe_delay = coalesce(visible_severe_delay, 0L),
    visible_carrier_delay_rate = coalesce(visible_carrier_delay_rate, visible_delay_rate),
    visible_carrier_n = coalesce(visible_carrier_n, 0L),
    vis_arr_delay_rate = coalesce(vis_arr_delay_rate, global_delay_rate),
    vis_arr_mean_delay = coalesce(vis_arr_mean_delay, 0)
  )

add_running_visible_features <- function(df) {
  df |>
    arrange(fl_date, crs_dep_time) |>
    group_by(fl_date) |>
    mutate(
      visible_count = row_number() - 1,
      visible_n_delayed = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
      visible_delay_rate = if_else(
        visible_count > 0,
        visible_n_delayed / visible_count,
        global_delay_rate
      )
    ) |>
    mutate(
      visible_recent_rate = purrr::map_dbl(
        row_number(),
        \(i) {
          if (i == 1) return(global_delay_rate)
          start_i <- max(1, i - 5)
          mean(replace_na(dep_del15[start_i:(i - 1)], 0))
        }
      )
```

```r
    ) |>
    mutate(
      visible_max_delay = purrr::map_dbl(
        row_number(),
        \(i) {
          if (i == 1) return(0)
          start_i <- 1
          vals <- dep_delay[start_i:(i - 1)]
          vals <- vals[!is.na(vals)]
          if (length(vals) == 0) 0 else max(vals)
        }
      ),
      visible_severe_delay = if_else(visible_max_delay > 60, 1L, 0L)
    ) |>
    ungroup() |>
    group_by(fl_date, op_unique_carrier) |>
    arrange(crs_dep_time, .by_group = TRUE) |>
    mutate(
      visible_carrier_n = row_number() - 1,
      visible_carrier_n_delayed = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
      visible_carrier_delay_rate = if_else(
        visible_carrier_n > 0,
        visible_carrier_n_delayed / visible_carrier_n,
        visible_delay_rate
      )
    ) |>
    ungroup() |>
    select(-visible_carrier_n_delayed)
}

train_fe <- add_running_visible_features(train_fe)
test_visible_fe <- add_running_visible_features(test_visible_fe)
# Option 1: carrier size group
# Collapses carriers by volume at PIT so new/rare carriers in 2025 fall into
# a meaningful bucket rather than being dropped or NA
carrier_size <- train |>
  count(op_unique_carrier) |>
  mutate(carrier_size = case_when(
    n >= 5000 ~ "large",
    n >= 1000 ~ "medium",
    TRUE ~ "small"
  )) |>
  select(op_unique_carrier, carrier_size)

# Option 2: hub vs. non-hub destination
# Major hub airports tend to have higher system-wide congestion; captures that
# without needing a dummy for every destination
hub_airports <- c("ATL", "ORD", "DFW", "DEN", "CLT", "JFK", "LAX",
                  "LGA", "EWR", "SFO", "SEA", "MIA", "BOS", "MSP", "DTW", "PHL")

# Option 3: regional grouping for destination
# Groups destinations by geography - more stable across years than individual airports
dest_region <- train |>
```

```r
  distinct(dest) |>
  mutate(dest_region = case_when(
    dest %in% c("BOS", "JFK", "LGA", "EWR", "PHL", "DCA", "IAD", "BWI") ~ "northeast",
    dest %in% c("ATL", "CLT", "MIA", "TPA", "MCO", "RDU", "BNA", "MEM") ~ "south",
    dest %in% c("ORD", "MDW", "DTW", "MSP", "CLE", "CMH", "IND", "MKE") ~ "midwest",
    dest %in% c("DFW", "DAL", "IAH", "HOU", "MSY", "SAT", "AUS") ~ "southwest",
    dest %in% c("DEN", "SLC", "PHX", "LAS", "ABQ") ~ "mountain",
    dest %in% c("LAX", "SFO", "SEA", "PDX", "SAN", "SJC", "OAK") ~ "west",
    TRUE ~ "other"
  ))

add_carrier_dest_features <- function(df) {
  df |>
    left_join(carrier_size, by = "op_unique_carrier") |>
    left_join(dest_region, by = "dest") |>
    mutate(
      carrier_size = coalesce(carrier_size, "small"),  # unseen carriers are small
      dest_region = coalesce(dest_region, "other"),  # unseen dests are other
      is_hub_dest = if_else(dest %in% hub_airports, 1L, 0L),
      carrier_size = factor(carrier_size, levels = c("small", "medium", "large")),
      dest_region = factor(dest_region),
      # binary flag derived from week rate
      high_disruption_calendar_day = if_else(
        week_delay_rate_smooth >= quantile(week_delay_rate_smooth, 0.90, na.rm = TRUE),
        1L, 0L
      )
    )
}

train_fe <- add_carrier_dest_features(train_fe)
test_visible_fe <- add_carrier_dest_features(test_visible_fe)
test_guess_fe <- add_carrier_dest_features(test_guess_fe)
model_vars <- c(
  "dep_del15",
  "sched_dep_hour",
  "sched_arr_hour",
  "is_late_arrival",
  "crs_elapsed_time",
  "carrier_size",
  "dest_region",
  "is_hub_dest",
  "distance",
  "distance_bucket",
  "month",
  "day_of_week",
  "peak_season",
  "is_evening",
  "is_morning",
  "carrier_delay_rate_smooth",
  "dest_delay_rate_smooth",
  "route_delay_rate_smooth",
  "prior_same_day_delay_rate",
  "before_today",
```

```
  "visible_delay_rate",
  "visible_count",
  "visible_max_delay",
  "visible_n_delayed",
  "visible_recent_rate",
  "visible_severe_delay",
  "visible_carrier_delay_rate",
  "visible_carrier_n",
  "flights_same_hour",
  "high_disruption_calendar_day"
)
```